ISLS Validation Harness

v1.0.0

Deterministic Benchmark, Empirical Validation
& Operational Metering Framework

This document specifies the complete testing, benchmarking, and operational validation layer that wraps the ISLS core. It is designed so that a single operator can:

(1) launch the system against live or historical data with one command,
(2) collect all metrics automatically,
(3) read a single dashboard to determine system health,
(4) identify exactly which subsystem needs iteration.

The harness itself is pure Rust, ships as two additional crates in the ISLS workspace, and requires zero ongoing human attention during normal operation.

Author: Sebastian Klemm

Date: 12 March 2026
Depends on: ISLS Rust Implementation Specification v1.0.0
New crates: `isls-harness` (C10), `isls-cli` (C11)

Operator workload target: <5 minutes/week for a healthy system. All anomalies surface automatically.

Contents

1 Architecture **2**
1.1 Operator Interaction Model . . . 2

2 Metric Taxonomy **2**
2.1 Layer Health Metrics (M1–M5) . . . 2
2.2 Core Quality Metrics (M6–M14) . . . 2
2.3 Performance Metrics (M15–M19) . . . 3
2.4 Empirical Domain Metrics (M20–M24) . . . 3

3 Benchmark Suite **3**
3.1 Benchmark Catalog . . . 4
3.2 Regression Tracking . . . 4

4 Validation Suite **4**
4.1 Validation Levels . . . 4
4.2 Formal Validation (V-Formal) . . . 5
4.3 Retrospective Validation (V-Retro) . . . 5
4.4 Live Validation (V-Live) . . . 5

5 Dashboard and Reporting **6**
5.1 One-Line Status . . . 6
5.2 Full Report . . . 6

6 Iteration Guidance Engine **6**
6.1 Diagnosis Rules . . . 6
6.2 Automated Iteration Report . . . 7

7 Data Sources and Ingestion Adapters **8**
7.1 Built-In Adapters . . . 8
7.2 Extension Point for Live Feeds . . . 8

8 Operational Modes **8**

9 File Layout and Persistence **8**

10 Configuration Extension **9**

11 Synthetic Ground-Truth Generator **9**

12 Implementation Contract for Claude Code **10**

1 Architecture

Two new crates extend the ISLS workspace:

```
isls/
  crates/
    isls-harness/    # C10: benchmark runners, validators, metric collectors, reporters
    isls-cli/        # C11: single-binary operator interface (commands, dashboard, config)
```

Design Rule

The harness never touches ISLS internals via back-doors. It uses exclusively the public API of `isls-engine` and `isls-archive`. If a metric cannot be computed from the public API, the public API must be extended — the harness must not circumvent encapsulation.

1.1 Operator Interaction Model

The entire system is operated through one binary: `isls`.

```
isls init                          # generate default config + data dirs
isls ingest <source>               # attach a live or file-based data source
isls run                           # start the macro-step loop (foreground or daemon)
isls run --replay <run_descriptor> # deterministic replay from saved descriptor
isls bench                         # run full benchmark suite, emit report
isls validate                      # run validation suite against collected data
isls report                        # print current health dashboard to stdout
isls report --json                 # machine-readable metrics export
isls report --html > report.html   # self-contained HTML dashboard
isls status                        # one-line system health summary
```

Target: A healthy system requires the operator to run `isls status` and glance at the output. Everything else is automated.

2 Metric Taxonomy

Every metric has: a unique ID, a human name, a computation method, a unit, a healthy range, and an alert threshold.

2.1 Layer Health Metrics (M1–M5)

One metric per ISLS layer, answering: "Is this layer functioning correctly?"

ID	Layer	Computation	Healthy	Alert
M1	L0 Observe	Ingestion rate: observations/sec over last 60s	> 0	$= 0$ for > 30s
M2	L1 Persist	Graph growth: $\Delta\|V\| + \Delta\|E\|$ per macro-window	≥ 0	Negative (data loss)
M3	L2 Extract	Active constraints: $\|S_{\text{active}}\|$ after scan	≥ 1	0 for > 1h
M4	L3 Consensus	Crystal rate: crystals committed per 24h	Profile-dep.	0 for $>$ configured silent period
M5	L4 Morph	Mutation rate: mutations per 24h	≥ 0	Negative invariant violation

2.2 Core Quality Metrics (M6–M14)

ID	Name	Computation	Healthy	Alert
M6	Replay fidelity	Run macro-step twice with saved RD; compare crystal digest sets	100% match	Any mismatch

ID	Name	Computation	Healthy	Alert
M7	Convergence rate	Wasserstein distance $\delta(w)$ between consecutive persistence diagrams	Decreasing	Increasing for > 5 windows
M8	Lattice stability	Mean free energy $\bar{F}$ of emitted crystals	< 0	≥ 0
M9	Gate selectivity	Fraction of macro-steps that pass Kairos gate	0.01–0.30	> 0.5 (too permissive) or 0 (too strict)
M10	Dual consensus MCI	Mean MCI of committed crystals	≥ 0.90	< 0.80
M11	PoR latency	Mean time from search→commit in PoR FSM	Profile-dep.	$> 2\times$ configured budget
M12	Evidence integrity	Fraction of crystals passing `verify_crystal`	100%	$< 100\%$
M13	Operator version drift	Count of unmatched operator versions in archive	0	> 0
M14	Storage efficiency	Cold-tier bytes per asset per month	< 3 MB	> 10 MB

2.3 Performance Metrics (M15–M19)

ID	Name	Computation	Healthy	Alert
M15	Macro-step latency	Wall-clock time per `macro_step()` call	< 60s	> 120s
M16	Memory footprint	RSS of process	< 4 GB (2k assets)	> 8 GB
M17	Extraction throughput	Constraint candidates evaluated per second	> 100	< 10
M18	Archive growth rate	Bytes appended to cold tier per 24h	Bounded	Unbounded growth
M19	Carrier migration latency	Time from migration trigger to stable new phase	< 5s	> 30s

2.4 Empirical Domain Metrics (M20–M24)

These measure how the system performs *in the real world* when connected to a domain profile (e.g., crypto markets via DSHAE).

ID	Name	Computation	Healthy	Alert
M20	Constraint hit rate	Fraction of emitted constraints that remain active 24h later	> 0.6	< 0.3
M21	Crystal predictive value	For crystals signaling structure X: did X persist?	> 0.5	< 0.3
M22	Signal lead time	Time between crystal emission and observable market event	> 0 (leading)	≤ 0 (lagging)
M23	Basket quality lift	DSHAE Sharpe ratio with ISLS basket vs. random basket	Positive	Negative
M24	Coverage growth	Number of tracked entities $\lvert V \rvert$ over time	Monotone increasing	Decrease (data loss)

3 Benchmark Suite

Invoked via `isls bench`. All benchmarks are deterministic (fixed seeds, synthetic data). Results are appended to a local benchmark history for regression tracking.

3.1 Benchmark Catalog

ID	Name	Procedure	Metric
B01	Ingestion throughput	Ingest 10^6 synthetic observations	obs/sec
B02	Graph update scaling	Update graph with $N = 100, 500, 2000, 5000$ vertices	μs per update vs. N
B03	Extraction scaling	Run `inverse_weave` on $N = 50, 200, 1000$ point clouds	ms per scan vs. N
B04	Cascade contraction	Apply DK→WF→TS→PC on synthetic 5D cloud	Contraction ratio, ms
B05	Dual consensus overhead	Run primal+dual paths; measure wall time vs. single	Overhead %
B06	Crystal serialization	Serialize+hash 10^4 crystals	μs per crystal
B07	Replay speed	Replay 1000 macro-steps from saved RD	steps/sec
B08	Evidence verification	Verify 10^4 crystals with $\bar{n} = 50$ evidence entries each	μs per verify
B09	Memory scaling	Run engine with $N = 100 \rightarrow 5000$ entities; measure RSS	MB vs. N
B10	Full macro-step	End-to-end `macro_step()` on reference dataset	ms, peak RSS

3.2 Regression Tracking

```rust
/// Benchmark result record
pub struct BenchResult {
    pub bench_id: String,
    pub timestamp: chrono::DateTime<chrono::Utc>,
    pub git_commit: String,
    pub metric_name: String,
    pub metric_value: f64,
    pub metric_unit: String,
    pub params: BTreeMap<String, String>,
}

/// Regression check: current vs. last 5 runs
pub fn check_regression(current: &BenchResult, history: &[BenchResult]) -> RegressionVerdict {
    let mean = history.iter().map(|h| h.metric_value).sum::<f64>() / history.len() as f64;
    let std = /* ... */;
    if current.metric_value > mean + 2.0 * std { RegressionVerdict::Regression }
    else if current.metric_value < mean - 2.0 * std { RegressionVerdict::Improvement }
    else { RegressionVerdict::Stable }
}
```

Output: `bench_history.jsonl` — one JSON line per benchmark run, enabling time-series analysis.

4 Validation Suite

Invoked via `isls validate`. Runs against *collected real data* (not synthetic). Answers: "Is the system producing correct and useful results in practice?"

4.1 Validation Levels

Three Validation Levels

V-Formal Invariant checks: all 20 ISLS invariants verified on every committed crystal in the archive. Deterministic, runs offline. `isls validate -formal`

V-Retro Retrospective accuracy: compare past crystal predictions against what actually happened. Requires ≥ 7 days of operational data. `isls validate -retro`

V-Live Live monitoring: continuous metric collection during `isls run`. Alerts on anomaly. Always-on, no separate command.

4.2 Formal Validation (V-Formal)

Iterate over every crystal in the archive and verify:

```
pub fn validate_formal(archive: &Archive, pinned: &BTreeMap<String,String>)
    -> FormalReport
{
    let mut report = FormalReport::new();
    for crystal in archive.all_crystals() {
        // V-F1: Content address integrity
        report.check("content_address", crystal.crystal_id == content_address(crystal));
        // V-F2: Evidence chain integrity
        report.check("evidence_chain", verify_evidence_chain(&crystal.evidence_chain).is_ok());
        // V-F3: Operator version match
        report.check("operator_versions", verify_operator_versions(crystal, pinned).is_ok());
        // V-F4: Gate values above thresholds
        report.check("gate_kairos", crystal.commit_proof.gate_values.kairos);
        // V-F5: Dual consensus
        let cr = &crystal.commit_proof.consensus_result;
        report.check("dual_consensus",
            cr.primal_score >= cr.threshold && cr.dual_score >= cr.threshold
            && cr.mci >= 0.80);
        // V-F6: PoR trace monotonicity
        report.check("por_trace", verify_por_trace(&crystal.commit_proof.por_trace));
        // V-F7: Free energy < 0
        report.check("free_energy", crystal.free_energy < 0.0);
        // V-F8: Append-only (no past crystal altered)
        report.check("immutability", !archive.was_modified(&crystal.crystal_id));
    }
    report
}
```

4.3 Retrospective Validation (V-Retro)

For each crystal that made a structural claim (e.g., "constraint $\mathfrak{g}_k$ binds assets A and B"), check whether the claim held true in subsequent data:

```
pub fn validate_retro(archive: &Archive, graph: &PersistentGraph,
    horizon: Duration) -> RetroReport
{
    let mut report = RetroReport::new();
    for crystal in archive.crystals_older_than(horizon) {
        for constraint in &crystal.constraint_program {
            // Re-evaluate constraint on data AFTER crystal emission
            let post_window = TimeWindow::after(crystal.created_at, horizon);
            let coverage = evaluate_constraint_coverage(constraint, graph, &post_window);
            report.add(RetroEntry {
                crystal_id: crystal.crystal_id,
                constraint_id: constraint.id,
                predicted_coverage: constraint.coverage,
                actual_coverage: coverage,
                still_active: coverage >= 0.5, // did the constraint survive?
            });
        }
    }
    // Aggregate: hit rate, mean coverage drift, false positive rate
    report.compute_aggregates();
    report
}
```

Key output: Constraint hit rate (M20), Crystal predictive value (M21), Signal lead time (M22).

4.4 Live Validation (V-Live)

Runs as a background task inside `isls run`. On every macro-step:

1. Compute all 24 metrics (M1–M24).
2. Append to `metrics.jsonl` (one JSON line per tick).
3. Check each metric against alert threshold.
4. If any alert fires: log `WARN` to structured log + write to `alerts.jsonl`.
5. Every N macro-steps (configurable, default 100): compute rolling aggregates and append to `summary.jsonl`.

5 Dashboard and Reporting

5.1 One-Line Status

isls status

```
$ isls status
ISLS v1.0.0 | UP 14d 3h | Entities: 847 | Edges: 12,403 | Crystals: 23
L0: OK (42 obs/s) | L1: OK (+12 edges/h) | L2: OK (31 active constraints)
L3: OK (1.3 crystals/day, MCI=0.94) | L4: OK (0 mutations pending)
Replay: PASS | Free Energy: -0.38 | Gate sel.: 0.07 | Storage: 2.1 GB
Health: GREEN
```

Color codes: **GREEN** = all metrics healthy. **YELLOW** = ≥ 1 metric in warning range. **RED** = ≥ 1 metric in alert.

5.2 Full Report

`isls report` outputs a structured report with sections:

1. **System Overview:** uptime, entity count, crystal count, storage usage.
2. **Layer Health:** M1–M5 with traffic-light indicators.
3. **Core Quality:** M6–M14 with sparkline trend (last 30 data points).
4. **Performance:** M15–M19 with p50/p95/p99 latencies.
5. **Empirical:** M20–M24 with rolling 7-day and 30-day aggregates.
6. **Alerts:** Active alerts with timestamp, metric ID, and current value.
7. **Iteration Guidance:** Ranked list of subsystems needing attention (see §6).

`isls report -html` emits a self-contained HTML file with embedded CSS and SVG charts (no JavaScript dependencies, no external fetches). Viewable in any browser.

`isls report -json` emits machine-readable JSON for programmatic consumption.

6 Iteration Guidance Engine

The harness does not just report metrics — it tells the operator *what to fix next*.

6.1 Diagnosis Rules

Trigger	Symptom	Diagnosis + Action	Priority
M6 $< 100\%$	Replay mismatch	Non-determinism in operator or serialization. Run `isls replay -diff` to identify first diverging macro-step. Check for `HashMap` usage, floating-point platform difference, or operator version drift.	P0
M8 ≥ 0	Positive free energy	Constraints too weak or temperature too high. Lower `extraction.alpha_min` or recalibrate temperature c_T.	P1
M9 > 0.5	Gate too permissive	Thresholds too low. Increase `thresholds.k` (crystal stability). Review whether noise is being crystallized.	P1
M9 $= 0$ for > 48h	Gate too strict	Thresholds too high for current data regime. Lower `thresholds.d` and `thresholds.q` by 10% increments.	P1

Trigger	Symptom	Diagnosis + Action	Priority
M10 < 0.80	Low MCI	Primal and dual paths diverge. Check operator cascade ordering. May indicate that DK contraction rate is too aggressive — reduce `dk_alpha`.	P1
M15 > 120s	Slow macro-step	Profile to find bottleneck: likely L2 extraction. Reduce `extraction.max_iterations` or `press_top_k`. Consider sparser constraint library.	P2
M20 < 0.3	Low constraint hit rate	Constraints are transient noise, not structural. Increase `extraction.bond_strength_min` so only durable constraints enter programs.	P1
M21 < 0.3	Low predictive value	Crystals do not predict future structure. Likely cause: macro-window too short. Increase `extraction.window_hours`.	P1
M22 ≤ 0	Lagging signals	ISLS detects structure *after* the market has moved. Reduce `por_T_min` and `por_T_stable` to allow faster PoR traversal.	P2
M12 $< 100\%$	Evidence corruption	Storage integrity failure. Run `isls validate -formal`. Identify corrupted tier segment. Re-derive from cold evidence or isolate.	P0

6.2 Automated Iteration Report

```rust
pub struct IterationItem {
    pub priority: Priority,        // P0, P1, P2
    pub metric_id: String,         // e.g. "M8"
    pub subsystem: String,         // e.g. "isls-extract"
    pub symptom: String,           // human-readable
    pub diagnosis: String,         // what's likely wrong
    pub action: String,            // what to change
    pub config_key: Option<String>, // which config field to adjust
    pub suggested_value: Option<String>, // suggested new value
}

pub fn generate_iteration_guidance(metrics: &MetricSnapshot, config: &Config)
    -> Vec<IterationItem>
{
    let mut items = Vec::new();
    if metrics.m6_replay_fidelity < 1.0 {
        items.push(IterationItem {
            priority: Priority::P0,
            metric_id: "M6".into(),
            subsystem: "isls-engine".into(),
            symptom: "Replay mismatch detected".into(),
            diagnosis: "Non-determinism in operator or serialization path".into(),
            action: "Run `isls replay --diff` to find first diverging step".into(),
            config_key: None,
            suggested_value: None,
        });
    }
    // ... (one rule per row in the diagnosis table above)
    items.sort_by_key(|i| i.priority.clone());
    items
}
```

The output of `isls report` includes an **"Action Items"** section listing these in priority order. The operator

reads top to bottom and addresses P0 first.

7 Data Sources and Ingestion Adapters

7.1 Built-In Adapters

Adapter	Source	Invocation
`file-csv`	CSV files (historical OHLCV)	`isls ingest -adapter file-csv -path data/`
`file-jsonl`	JSONL observation logs	`isls ingest -adapter file-jsonl -path obs.jsonl`
`replay`	Saved RunDescriptor	`isls run -replay run_desc.json`
`synthetic`	Deterministic synthetic generator	`isls ingest -adapter synthetic -entities 500`

`synthetic` generates deterministic data with configurable correlation structure, regime switches, and known constraint programs — enabling ground-truth validation (the harness *knows* which constraints exist and can measure detection accuracy exactly).

7.2 Extension Point for Live Feeds

```rust
/// Trait for live data source adapters (domain-specific, out of ISLS core scope)
pub trait LiveAdapter: ObservationAdapter {
    fn connect(&mut self) -> Result<()>;
    fn poll(&mut self) -> Result<Vec<Vec<u8>>>;  // raw observation bytes
    fn disconnect(&mut self);
}
// A crypto-market adapter implementing this trait would connect to exchange WebSockets.
// The harness provides the framework; the domain-specific adapter is a plugin.
```

8 Operational Modes

Four Operational Modes

Bench	Synthetic data, deterministic seeds. Measures performance. No side effects. `isls bench`
Validate	Real archived data, offline. Measures correctness and predictive quality. `isls validate`
Shadow	Live data ingestion, full pipeline, but **no downstream action**. Crystals are emitted and archived but not forwarded to DSHAE. Enables safe burn-in. `isls run -mode shadow`
Live	Full operation. Crystals forwarded to downstream consumers. Metrics collected. `isls run -mode live`

Recommended deployment sequence:

1. Run `isls bench` after every code change. Ensure no regressions.
2. Deploy in `shadow` mode for $\geq$ 14 days. Collect metrics.
3. Run `isls validate -retro` after 14 days. Check M20–M24.
4. If all GREEN: switch to `live` mode.
5. Run `isls report` weekly. Address iteration items top-down.

9 File Layout and Persistence

```
~/.isls/
  config.json              # master config (ISLS + harness)
  data/
    hot/                   # hot-tier observation cache (7 days)
    warm/                  # warm-tier compressed archive (90 days)
    cold/                  # cold-tier indefinite evidence
    crystals/              # committed crystal archive (one file per crystal)
  metrics/
    metrics.jsonl          # per-tick metric log (M1-M24)
    alerts.jsonl           # fired alerts
    summary.jsonl          # rolling aggregates (every 100 ticks)
```

```
    bench_history.jsonl    # benchmark regression history
  reports/
    latest.html            # most recent HTML report
    latest.json            # most recent JSON report
  replay/
    *.run_descriptor.json  # saved run descriptors for replay
```

10 Configuration Extension

The harness adds a `harness` section to the ISLS config:

```
harness:
  metrics_interval_steps: 1        # collect metrics every N macro-steps
  summary_interval_steps: 100      # rolling summary every N steps
  alert_cooldown_seconds: 300      # min time between repeated alerts
  bench:
    synthetic_entities: 500
    synthetic_windows: 1000
    seed: 42
  validate:
    retro_horizon_days: 7
    formal_on_startup: true        # run formal validation on every startup
  report:
    auto_generate_hours: 24        # auto-generate HTML report every N hours
    retention_days: 90             # keep old reports for N days
  iteration:
    enabled: true                  # auto-generate iteration guidance
    p0_halt: false                 # if true, halt engine on P0 alert (safety mode)
```

11 Synthetic Ground-Truth Generator

The most powerful validation tool: a deterministic data generator that embeds *known* constraint programs into synthetic data, then measures ISLS's ability to recover them.

```rust
pub struct SyntheticScenario {
    pub name: String,
    pub entities: usize,
    pub windows: usize,
    pub seed: u64,
    pub planted_constraints: Vec<PlantedConstraint>,
    pub regime_switches: Vec<(usize, RegimeType)>,  // (window, new_regime)
}

pub struct PlantedConstraint {
    pub template: ConstraintTemplate,
    pub parameters: BTreeMap<String, f64>,
    pub active_from: usize,   // window number
    pub active_until: usize,
    pub strength: f64,        // how strongly the constraint is enforced in data
}

/// Generate synthetic observations with planted constraints
pub fn generate(scenario: &SyntheticScenario) -> Vec<Vec<Observation>> { /* ... */ }

/// Score ISLS's recovery of planted constraints
pub fn score_recovery(planted: &[PlantedConstraint], discovered: &ConstraintProgram)
    -> RecoveryScore
{
    // Precision: fraction of discovered constraints that match a planted one
    // Recall: fraction of planted constraints that were discovered
    // F1: harmonic mean
    // Parameter accuracy: mean |param_discovered - param_planted| for matched constraints
    RecoveryScore { precision, recall, f1, param_accuracy }
}
```

Reference scenarios:

1. **S-Basic:** 50 entities, 3 planted Band constraints, no regime switches.
2. **S-Regime:** 200 entities, 5 constraints, 2 regime switches (calm→volatile→calm).

3. **S-Causal:** 100 entities, 3 Granger causal chains $A \to B \to C$, planted with 2h lag.
4. **S-Break:** 200 entities, 4 constraints active for 500 windows then breaking.
5. **S-Scale:** 2000 entities, 20 constraints, full regime complexity.

`isls bench` runs all five scenarios and reports precision/recall/F1 for each.

12 Implementation Contract for Claude Code

Build Contract

1. Create crate `isls-harness` with modules: `metrics`, `bench`, `validate`, `synthetic`, `report`, `iterate`.
2. Create crate `isls-cli` with binary `isls` implementing all commands from §1.
3. Implement all 24 metrics (M1–M24) with collection, alert, and persistence.
4. Implement all 10 benchmarks (B01–B10) with regression tracking.
5. Implement V-Formal, V-Retro, V-Live validation levels.
6. Implement the 10 diagnosis rules from §6 and the iteration guidance generator.
7. Implement the synthetic ground-truth generator with all 5 reference scenarios.
8. Implement `isls report -html` with self-contained HTML (inline CSS, SVG sparklines, no external deps).
9. All tests: `cargo test -workspace` must pass.
10. The binary `isls status` must produce output matching the format in §5.1 within 1 second.

Prompt for Claude Code

```
Build isls-harness and isls-cli as specified in ISLS_ValidationHarness_v1_0_0.tex.
24 metrics, 10 benchmarks, 3 validation levels, 5 synthetic scenarios,
10 diagnosis rules, HTML report generator. cargo test --workspace must pass.
The spec is the single source of truth.
```

End of specification — ISLS Validation Harness v1.0.0

24 metrics. 10 benchmarks. 3 validation levels. 10 diagnosis rules. 5 synthetic scenarios.

One binary. One glance.